

Universidade Federal do Rio Grande do Norte
Sistemas Big Data DIM0198 - 2026.2

Relatório de Implementação de DBSCAN em MPI

Eduardo Nestor Ferreira Marinho
Kael Amir Pina Paraguassu Silva
Matheus Leão de Carvalho
Samuel Morais da Rocha e Silva

Natal, November 5, 2026.

1 Implementação

1.1 DBScan

A implementação do algoritmo DBSCAN foi dividida em duas fases principais: o mapeamento de vizinhança e a expansão dos agrupamentos (*clusters*).

Na primeira etapa (`calcular_vizinhos`), o algoritmo mede a distância euclidiana entre os pontos em d dimensões e guarda quem está no ε (`eps`) máximo. O próprio ponto entra na lista. Para otimizar o processamento e facilitar a comunicação em rede, os vizinhos identificados para cada ponto são agrupados em um único vetor contíguo na memória (`plano`), em vez de serem armazenados em listas encadeadas independentes. Além disso, o vetor é acompanhado por um outro vetor auxiliar que registra a quantidade de vizinhos de cada ponto (`contagens`).

Na segunda etapa (`expandir`), os pontos são classificados a partir da densidade de sua vizinhança. Todos os registros iniciam rotulados como ruído (`-1`). O algoritmo então percorre a base em busca de pontos centrais, os que têm ao menos `min_pts` vizinhos nessa lista. Sempre que acha um ponto central ainda sem rótulo, abre um agrupamento e o expande em largura: vizinho de ponto central entra no grupo e, se também for central, a busca continua a partir dele.

1.2 Distribuição

Como a busca de vizinhos compara cada ponto com toda a base, o MPI divide essa carga em fatias. Todos os processos calculam a sua parte, inclusive o de *rank* 0:

1. **Leitura e Difusão (*Broadcast*):** O processo de *rank* 0 carrega o arquivo binário com a matriz normalizada e envia as dimensões e a matriz inteira aos demais com `MPI_Bcast`.

2. **Particionamento e Busca Paralela:** O conjunto total de n pontos é dividido de forma balanceada em fatias contíguas entre os processos disponíveis. Cada processo calcula, de maneira independente e sem necessidade de comunicação intermediária, as listas de vizinhos apenas para os pontos pertencentes à sua fatia local.

3. **Consolidação (*Gather*):** Os processos trocam o tamanho das listas (`MPI_Allgather`) e devolvem as contagens e os vizinhos ao processo 0 com `MPI_Gatherv`.

4. **Expansão Sequencial:** Com as listas reunidas, só o processo 0 executa `expandir`. O resultado não depende do número de processos, porque essa etapa é sempre a mesma.

O monitoramento de desempenho é realizado com barreiras de sincronização (`MPI_Barrier`) e cronometragem (`MPI_Wtime`), isolando os tempos gastos na busca paralela (consolidado pelo maior tempo entre os processos via `MPI_Reduce`), na junção dos dados pela rede e na expansão final.

1.3 Job

Os testes de desempenho e escalabilidade foram automatizados no gerenciador de filas Slurm por meio do script `job_dbscan.sh`. O ambiente de execução foi dimensionado na

partição `intel-128` com a alocação de 16 nós computacionais e até 32 tarefas por nó (1 CPU por tarefa), totalizando até 512 processos simultâneos e tempo limite de 30 minutos.

A bateria experimental executou o algoritmo sobre o arquivo `dados/amostra.bin` utilizando os parâmetros fixos $\varepsilon = 0{,}065$ e MinPts $= 10$. Para avaliar o comportamento do código em diferentes escalas de paralelismo, o número de processos (P) foi variado em potências de dois ($1, 2, 4, 8, 16, 32, 64, 128, 256$ e 512), realizando-se 3 repetições consecutivas para cada configuração a fim de obter as médias.

1.4 Saídas

Ao término de cada execução, o processo mestre consolida as métricas e gera os artefatos de saída na pasta `resultados/`:

- **Métricas de Desempenho (`tempos.csv`):** Arquivo tabular incrementado a cada execução com o número de processos, o índice da repetição, o tempo total, os tempos parciais de cada etapa (busca, junção e expansão) e os totais de agrupamentos e pontos de ruído.
- **Rótulos de Agrupamento (`rotulos_p<nprocs>.bin`):** Arquivo binário contendo o vetor de classes atribuídas a cada um dos n pontos da amostra, utilizado posteriormente pelos scripts de visualização.
- **Resumo da Execução (`resumo.json`):** Gerado na execução de referência com 1 processo, documenta os hiperparâmetros utilizados, a quantidade de agrupamentos e ruídos encontrados e o tamanho dos 10 maiores *clusters*.

2 Resultados

2.1 Desempenho

2.1.1 Eficiência, *speedup* e tempos de execução

A tabela pura produzida em formato CSV foi agregada com o objetivo de resumir as métricas de tempo de execução em relação às três execuções para cada configuração de número de processos. O resultado é uma tabela com os seguintes atributos:

- N_p (Índice): O número de processos paralelos utilizados na execução do algoritmo.
- $\bar{T}_{total}$: O tempo médio total decorrido para a execução completa do algoritmo considerando as repetições (em segundos).
- $\bar{T}_{buscar}$: A média do maior tempo gasto na etapa de busca de vizinhos dentro do algoritmo (em segundos). Utilizamos o maior tempo para a operação de busca entre os nós por este ser o gargalo da execução distribuída.
- $\bar{T}_{gather}$: O tempo médio gasto no processo de consolidação de dados paralelos (operação *gather*) (em segundos).
- $\bar{T}_{expandir}$: O tempo médio gasto especificamente na expansão dos clusters paralelos (em segundos).

Table 1: Métricas médias de desempenho por número de processos paralelo.

N_p	$\bar{T}_{total}$	$\bar{T}_{buscar}$	$\bar{T}_{gather}$	$\bar{T}_{expandir}$
1	1.4196	1.4107	0.0049	0.0039
2	0.6940	0.6839	0.0052	0.0043
4	0.3575	0.3454	0.0070	0.0037
8	0.2034	0.1889	0.0084	0.0037
16	0.1143	0.0954	0.0117	0.0039
32	0.1457	0.1015	0.0175	0.0056
64	0.1334	0.0542	0.0362	0.0060
128	0.1498	0.0293	0.0448	0.0062
256	1.1404	0.0162	1.1115	0.0060
512	1.1314	0.0085	0.9416	0.0064

Table 2: *Speedup* e Eficiência médias obtidas para o tempo total de execução.

N_p	$\bar{S}$	$\bar{E}$
1	1.0000	1.0000
2	2.0455	1.0227
4	3.9709	0.9927
8	6.9798	0.8725
16	12.4208	0.7763
32	9.7427	0.3045
64	10.6404	0.1663
128	9.4779	0.0740
256	1.2448	0.0049
512	1.2547	0.0025

A Tabela 1 consolida os resultados obtidos.

Para uma análise mais aprofundada, foram calculados os valores de *speedup* e eficiência médios, tomando como base os valores de $\bar{T}_{total}$ para as diferentes configurações de números de processos.

As valores da análise estão visíveis na Tabela 2, e organizados graficamente na Figura 1. É possível observar o decaimento do *speedup* em relação ao comportamento ótimo - a manutenção do crescimento linear - a partir da distribuição em 4 processos, ponto no qual o tempo dedicado à operação de *gather* aumenta sua taxa de crescimento, como pode ser visto em seguida na Figura 2.

Vale apontar que, de acordo com a progressão da eficiência calculada na Figura 1, o processamento chega ao patamar de execução eficiência super-linear, com o aproveitamento de recursos de hardware ultrapassando a expectativa linear ótima de $\bar{E} = 1$.

De posse da Tabela 1, foi construído um gráfico de linhas na Figura 2 detalhando, para cada número de processos, o tempo médio total da execução da rotina e das operações que a compõe.

Dentre pontos de observação valiosos do gráfico, destacam-se:

- O tempo decorrido médio para a operação de *gather* cresce em picos. Até os 32 processos, o custo desta etapa evolui de maneira quase linear, mas, a partir deste

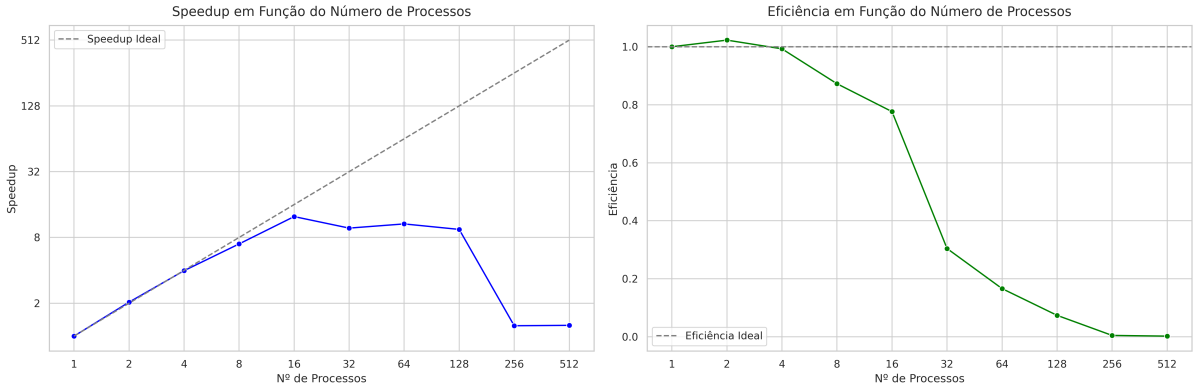


Figure 1: Gráficos de *speedup* e eficiência médios.

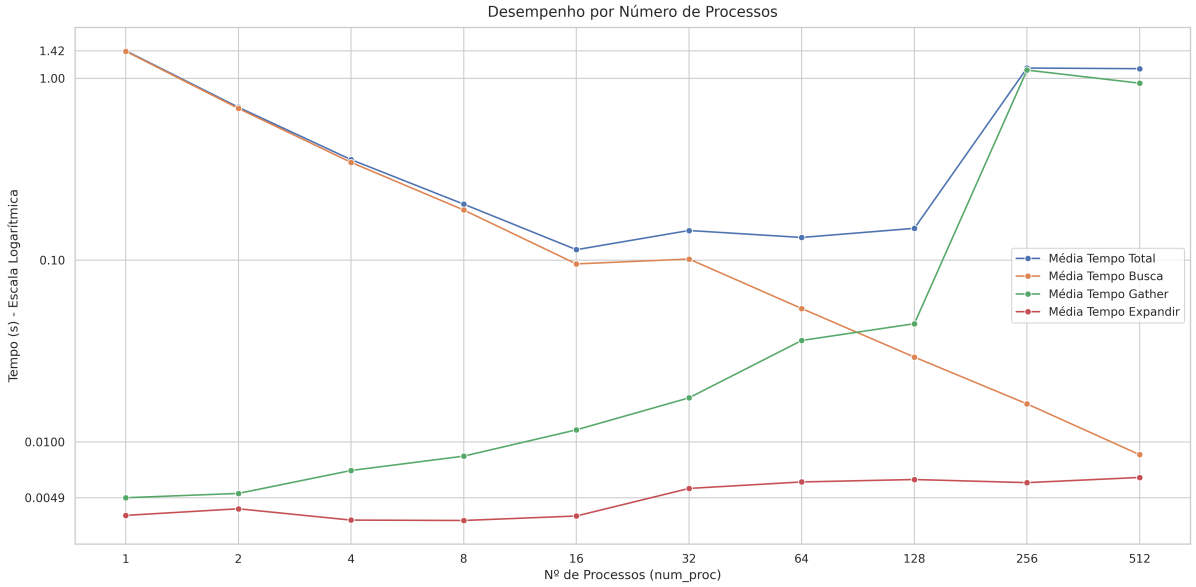


Figure 2: Gráfico de tempo de execução do DBScan.

ponto, o crescimento é dado em saltos, sendo a transição dos 128 aos 256 processos o mais vertiginoso. Este comportamento deve-se à necessidade de comunicação entre diferentes nós da rede, já que a partição `intel-128` usada para a execução do *job* possui processador com 32 núcleos. O trabalho, que antes poderia ser concentrado em um único ambiente, precisa ser distribuído pela rede, adicionando um componente de latência de comunicação. Além da latência de rede, a operação de *gather* no MPI implica no nó mestre receber volumes de dados dos demais; para um número elevado de processos, é possível haver saturação de rede I_O.

- O tempo médio total da execução do algoritmo cresce de maneira inexpressiva de 256 a 512 processos, com o tempo referente ao *gather* diminuindo entre as duas execuções. Análises mais aprofundadas sobre a infraestrutura física das partições utilizadas e dos parâmetros do *job* podem revelar relações que expliquem este comportamento.

- O tempo decorrido médio para a operação de busca decai de forma quase linear com a proliferação dos processos. Isso ocorre pois esta operação é **local**, não envolvendo

trocas de mensagem na rede. Ou seja, esta operação beneficia-se grandemente pelo processamento distribuído, já que a quantidade de pontos processados em cada instância reduz com a adição de processos, diminuindo, também, o esforço de cada processador.

2.2 *Clustering*

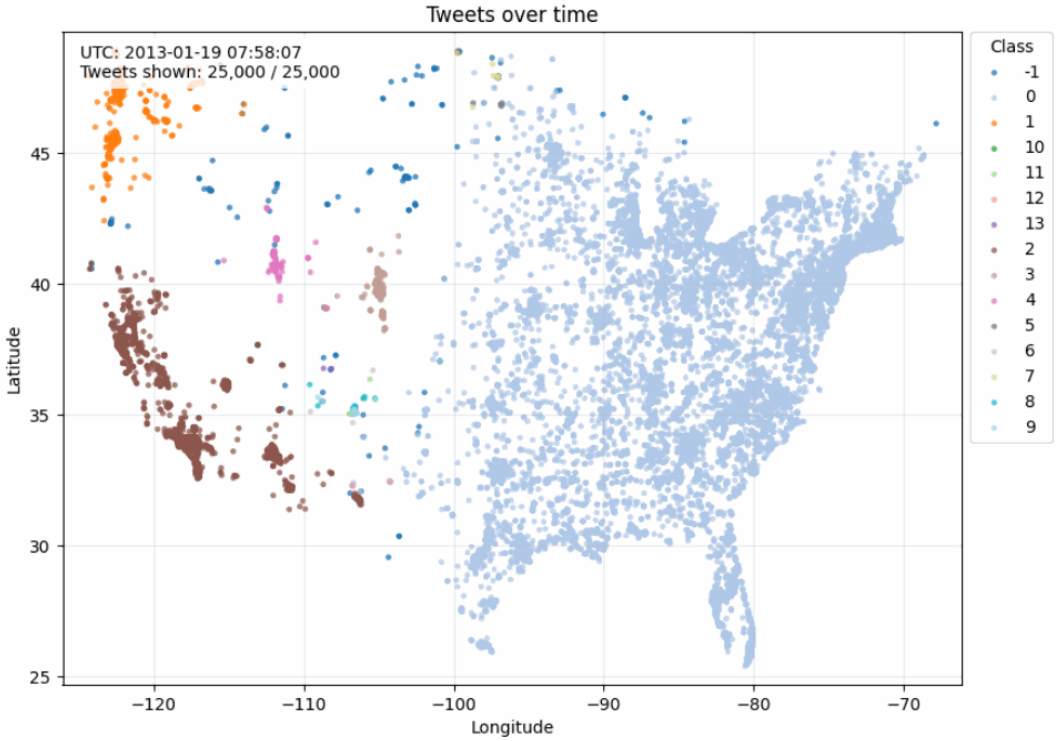


Figure 3: Exemplo de clustering.